

A PBL-III Report

On

Carbon Emission Prediction in Data Centres Using Deep Recurrent Neural Networks Optimised by Ninja Optimisation Algorithm

Submitted to Manipal University Jaipur

Towards the partial fulfillment for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In Computers Science and Engineering

2025-2026

By

Anwesha Singh

23FE10CSE00299

V-Semester



**MANIPAL UNIVERSITY
JAIPUR**

Under the guidance of

Dr Shishir Singh Chauhan

Name of the Supervisor

Signature of Supervisor

Department of Computer Science and Engineering

School of Computer Science and Engineering

Manipal University Jaipur

July-Dec 2025

Jaipur, Rajasthan - 303007

ACKNOWLEDGEMENT

This project would not have been completed without the help, support, comments, advice, cooperation and coordination of various people. However, it is impossible to thank everyone individually; I am hereby making a humble effort to thank some of them.

I acknowledge and express my deepest sense of gratitude to my internal supervisor Dr. Shishir Singh Chauhan for his constant support, guidance, and continuous engagement. I highly appreciate his technical comments, suggestions, and criticism during the progress of this project titled “Carbon Emission Prediction in Data Centres Using Deep Recurrent Neural Networks Optimised by Ninja Optimisation Algorithm”.

I owe my profound gratitude to Dr. Neha Chaudhary, Head, Department of CSE, for her valuable guidance and for facilitating me during my work. I am also very grateful to all the faculty members and staff for their precious support and cooperation during the development of this project.

Finally, I extend my heartfelt appreciation to my classmates for their help and encouragement.

Anwesha Singh

23FE10CSE00299



**MANIPAL UNIVERSITY
JAIPUR**
(University under Section 2(f) of the UGC Act)

Department of Computer Science and Engineering

School of Computer Science and Engineering

Date: 18.11.2025

CERTIFICATE

This is to certify that the project entitled “Carbon Emission Prediction in Data Centres Using Deep Recurrent Neural Networks Optimised by Ninja Optimisation Algorithm)” is a bonafide work carried out as PBL-3 End Term Assessment in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, by Anwesha Singh bearing registration number 23FE10CSE00299, during the academic semester V of year 2025-2026.

Place: Manipal University Jaipur, Jaipur

Name of the project guide: Dr. Shishir Singh Chauhan

Signature of the project guide: _____

ABSTRACT

The rapid increase in data centres worldwide has led to significant increase in energy consumption and corresponding carbon emissions, which makes environmental sustainability a critical concern for IT and computing industry. This project aims to propose a comprehensive approach to predicting the carbon emissions from data centre operations using time series sensor data and advanced deep learning architectures. To be specific, I am developing stacked and bidirectional Long Short Term Memory recurrent neural network optimised using Ninja Optimisation Algorithm, which is a recently developed metaheuristic optimisation technique inspired by tactical nature of Ninjas. The dataset comprises observations from data centre sensors capturing electrical metrics, temperature, and usage patterns. Through preprocessing, feature engineering, and rigorous experimental validation, my proposed model achieves validation mean absolute error 1.2128 and test mean absolute error of 1.7302, outperforming baseline models optimised using other optimisation algorithms (like Optuna). The results show that hyperparameter tuning via Ninja Optimisation Algorithm yields strong and robust generalisation, faster convergence, and superior predictive performance for energy forecasting tasks.

1. INTRODUCTION

1.1 Motivation

The increasing dependence on digital infrastructure, cloud computing, and large scale data centres has led to exponential demand for electrical energy. Data centres, while enabling modern services, are responsible for approximately 1 to 2% of global electricity consumption, which translates to significant amount of carbon emissions. Many organisations and researchers are pushing towards data driven approach to monitor and predict carbon emissions and environmental footprint of these data centres. Accurate forecasting of energy consumption and associated carbon emissions is important not only for compliance with environmental regulations but also for operational efficiency and cost savings.

Traditional statistical forecasting methods cannot capture the complex, nonlinear temporal dependencies in the data centre sensor readings. Deep learning, especially recurrent neural networks such as Long Short Term Memory architectures, have been proving themselves as highly effective for sequential pattern recognition in diverse domains. The success of these models heavily relies on appropriate selection of hyperparameters, including number of layers, dropout rate, learning rate, and batch size. Manual tuning is time consuming, inefficient, and often leads to suboptimal configurations.

This project is motivated by the need to combine the power of deep recurrent neural networks with intelligent, automated hyperparameter optimisation. I adopted Ninja Optimisation Algorithm, a nature inspired metaheuristic that balances explorations and exploitation during search, as a novel approach to finding the optimal configuration for our time series influenced carbon emission forecasting task. Using this combination, we aim to build a model that accurately predicts carbon emissions from minute-by-minute sensor data, hence supporting proactive decision making for energy management in data centres.

2. LITERATURE REVIEW

2.1 Review of Prior Work

Classical machine learning methods like support vector machines and decision trees have been widely used for energy forecasting in data centres. However, these approaches fail to capture temporal dependencies inherent in sequential energy data, where current consumption depends heavily on historical patterns. Recurrent Neural Networks, particularly Long Short Term Memory (LSTM) networks, have emerged as more suitable for this task due to their ability to learn long-term dependencies through gating mechanisms. Bidirectional LSTM variants, which process data in both forward and backward directions, further improve performance by incorporating contextual information from both temporal directions, achieving R^2 values exceeding 0.90 on real-world energy datasets. [5]

Contemporary research by Ben Ghorbal and colleagues has introduced Dual-Path Recurrent Neural Networks that simultaneously capture short-term oscillations and long-term trends. These architectures have achieved state-of-the-art performance on industrial emission prediction, with R^2 scores of 0.9736 on cement production emissions—a domain similar to data centre energy forecasting in terms of complexity. [1]

Despite the advantages of deep learning models, their practical effectiveness depends critically on hyperparameter selection. Traditional methods like grid search are computationally expensive, whilst Bayesian optimisation (such as Optuna) may converge to local optima. The Ninja Optimisation Algorithm (NiOA), inspired by strategic ninja tactics, offers a promising alternative by adaptively balancing exploration and exploitation, proving particularly effective for high-dimensional neural network hyperparameter spaces. [2]

My prior work involved implementing LSTM models with Optuna-based tuning for data centre carbon emission prediction. This research extends that foundation by incorporating NiOA, bidirectional stacked LSTM architectures, and robust checkpoint mechanisms for cloud computing environments.

2.2 Comparative Analysis of Techniques

Table 1: A comparative summary of various optimisation techniques evaluated in recent studies:

Optimisation Method	Strengths	Limitations
Grid Search	Exhaustive, deterministic	Computationally prohibitive for large spaces
Random Search	Efficient for high-dimensional spaces	No convergence guarantee
Bayesian Optimisation	Principled probabilistic approach	May converge to local optima

Genetic Algorithm	Good global exploration	High computational overhead
Particle Swarm Optimisation	Effective for continuous spaces	Requires careful parameter tuning
Harris Hawks Optimisation	Adaptive search strategy	Limited theoretical analysis
Ninja Optimisation Algorithm	Adaptive balance, fast convergence, superior performance	Recently introduced, ongoing theoretical work

Research shows that NiOA-optimised Dual-Path RNNs achieved $R^2 = 0.9736$ for CO₂ emission forecasting, outperforming Sine Cosine Algorithm ($R^2 = 0.9715$), Harris Hawks Optimisation ($R^2 = 0.9613$), and JAYA algorithm ($R^2 = 0.9520$). Statistical validation through ANOVA and Wilcoxon tests confirmed NiOA's superiority with high significance. [1]

2.3 Outcome of Literature Review

Several key findings emerge from the readings:

First, LSTM architectures are well-suited for time series prediction of energy and carbon emissions from data centre sensors. Bidirectional variants enhance performance by incorporating contextual information, whilst stacked architectures enable hierarchical feature extraction across multiple temporal scales.

Second, hyperparameter selection critically affects deep learning model performance. Conventional optimisation methods are either computationally expensive or prone to local optima convergence, limiting practical deployment of sophisticated models.

Third, the Ninja Optimisation Algorithm balances broad exploratory search with focused refinement, making it particularly effective for high-dimensional neural network optimisation problems.

Fourth, empirical evidence demonstrates that NiOA-optimised recurrent networks achieve state-of-the-art performance on emission prediction tasks, with validation metrics significantly exceeding competing approaches. [2][3]

Finally, a research gap exists wherein bidirectional stacked LSTM networks optimised through NiOA have not been extensively applied to real-time data centre carbon emission prediction with comprehensive validation. Additionally, practical considerations regarding checkpoint management and session resilience in cloud computing remain underexplored.

This research addresses these gaps by developing and rigorously evaluating an NiOA-optimised bidirectional stacked LSTM model for data centre carbon emission prediction, with emphasis on practical robustness through checkpoint mechanisms that enable recovery from session interruptions—a valuable contribution to deploying long-running hyperparameter optimisation processes in cloud environments.

3. PROBLEM STATEMENT

Conventional carbon footprint estimation methods are inaccurate and fail to capture dynamic, time-evolving power consumption patterns in data centers. Despite the availability of advanced deep learning architectures and optimization frameworks, accurate, real-time prediction of carbon emissions in data centers remains challenging because of high dimensional sensor data, complex temporal dynamics, and difficulty of hyperparameter tuning. Manual hyperparameter tuning is inefficient and suboptimal. Hence, a real-time, scalable, and accurate predictive system is essential for optimizing energy use and minimizing carbon emissions in data centers.

4. RESEARCH OBJECTIVES

- To preprocess, scale, and engineer features from multi sensor data centre datasets for time series forecasting.
- To design and implement a stacked, bidirectional Long Short Term Memory recurrent neural network capable of capturing complex temporal dependencies when predicting carbon emissions.
- Implement automated hyperparameter optimization to eliminate manual human intervention.
- To evaluate the optimised model's performance against baseline approaches and analyse convergence behaviour, generalisation ability, and prediction accuracy.
- To interpret results, discuss implications for sustainable data centre operations, and outline future directions for model enhancement and deployment.

5. METHODOLOGY AND FRAMEWORK

5.1 System Architecture

The system architecture so far comprises of several stages like: Data preprocessing, feature engineering, sequence generation, hypermeter optimisation, model training, and evaluation.

Data is first loaded from a comprehensive dataset capturing electrical voltage, current, power, frequency, energy, temperature, and usage metrics from multiple data centre devices over extended periods. After performing data quality checks, time stamp conversion, duplicate removal, and outlier detection using z score thresholding.

For the sequence modelling step, sliding window approach is adopted, creating a sequence of 10 consecutive timesteps as inputs and the **subsequent timestep as the target output**. This approach aligns with established practices in time series prediction and enables recurrent network to learn temporal dependencies over short periods. This approach follows the commonly used method for predicting values from time-based data and helps the network understand patterns that happen over short periods.

The Ninja optimisation is then invoked to search the hypermeter space. During this step a population of candidate solutions is initialised termed as agents, each representing a distinct hypermeter configuration. Through multiple iterations, agents go on to **explore**, where many different configurations are sampled, and then exploitation, where the configurations are refined around the best performing solutions found. The fitness of each agent is evaluated by training a Long Short Term Memory model on training set and measuring validation loss. Finally, the configuration yielding the lowest validation loss is selected for model training.

5.2 Algorithms and Techniques

5.2.1 Long Short Term Memory Recurrent Neural Networks

Long Short Term Memory networks are a specialised variant of recurrent neural networks that are designed to address the vanishing gradient problem inherent in standard recurrent architectures. They incorporate gating mechanisms comprising input, forget, and output valves that regulate information flow, allowing the network to learn and retain dependencies over extended sequences. The architecture of this model uses a bidirectional Long Short Term Memory layer as the first layer, allowing the model to process sequences in both forward and reverse temporal directions, thus capturing richer contextual information (past and future values both, giving 2x information to learn).

Subsequent layers consist of stacked Long Short Term Memory units, the number of which is determined through optimisation. Dropout regularisation is applied after each Long Short Term Memory layer to mitigate overfitting by randomly deactivating neurons during training process. Batch normalisation is utilised to normalise activations, stabilise training dynamics, and accelerate the convergence. Global Max Pooling is employed to extract salient features across the sequence, followed by dense layers that perform regression to predict the continuous valued target representing energy which translates to carbon emission.

5.2.2 Ninja Optimisation Algorithm

The Ninja Optimisation Algorithm is a metaheuristic inspired by the strategic behaviour of ninjas, balancing reconnaissance (observation of region to locate a target) across unknown regions with focused, precise actions near identified targets.

It uses following key steps:

Population Initialisation: A set of agents is randomly initialised, each agent representing a unique combination of hyperparameters.

Fitness Evaluation: Each agent's hyperparameter configuration is used to train a Long Short Term Memory model, and validation loss is computed as the fitness measure.

Exploration Phase: In early rounds of optimisation process, agents explore a wide range of combinations hyperparameter values. This phase prioritises diversity (exploring big search space), allowing agents to investigate vast regions of the search space, hence reducing the likelihood of premature convergence and missing a better solution.

Exploitation Phase: In later rounds of optimisation, agents transition towards exploitation, concentrating their search around the best-known configurations. Small tweaks, incremental adjustments are made to refine hyperparameters and maximise the predictive performance.

Adaptive Phase Selection: The algorithm tunes the balance between exploration and exploitation, dynamically adjusts the probability of exploration versus exploitation throughout the optimisation process, mimicking tactical approach of ninjas. It ensures a smooth transition from broad search to targeted refinement.

Greedy Selection: Agents retain a new solution only if their fitness (validation loss) improves over their previous configuration.

The algorithm concludes after a predefined number of iterations or upon achieving satisfactory convergence, returning the best found hyperparameter configuration.

5.2.3 Model Training and Validation

Data is partitioned into training, validation, and test sets in a 70:15:15 ratio. During Ninja Optimisation Algorithm based hyperparameter search, each candidate model is trained on the training set for a maximum of 30 epochs with early stopping triggered if validation loss does not improve for 5 consecutive epochs. Once the optimal hyperparameters are identified, the final model is trained for up to 200 epochs (with early stopping) with patience of 15 epochs for early stopping and learning rate reduction on plateau to ensure convergence.

Performance is assessed using Mean Absolute Error and Root Mean Squared Error on both validation and test sets. These metrics quantify the average magnitude of prediction errors, with lower values indicating better model performance. Training and validation loss curves are plotted to diagnose overfitting and convergence behaviour.

5.3 Detailed Design Methodologies

5.3.1 Data Preprocessing

Raw sensor data is first screened and filtered for completeness and consistency. Missing values are forward filled, preserving temporal continuity. Duplicate records are removed to prevent bias in training. Outliers are detected using a z score threshold of 3, and corresponding

observations are excluded to ensure robust model training. Lag features are generated to incorporate historical context, and temporal attributes such as hour and day_of_week are extracted to capture cyclical patterns in data centre operations.

5.3.2 Feature Scaling and Encoding

All continuous numerical features are standardised to zero mean and variance of 1, ensuring that the optimisation landscape is well conditioned. Categorical variables are one hot encoded, converting discrete labels into binary indicator variables suitable for neural network processing.

5.3.3 Sequence Generation

A sliding window mechanism constructs sequences of length 10, each having multi-dimensional feature vectors from consecutive timesteps. The target for each sequence is the energy (which translates to carbon emission) at the subsequent time step, framing the problem as supervised sequence to value regression.

5.3.4 Hyperparameter Optimisation Configuration

The hyperparameter search space includes the number of Long Short Term Memory layers, ranging from 2 to 3. The number of units per layer, chosen from 64 or 128; dropout rate, varied between 0.3 and 0.6; optimizer type, set to AdamW (was also chosen by Optuna algorithm) for its superior generalisation properties. Batch size, fixed at 32 for computational efficiency. The Ninja Optimisation Algorithm initialises 12 agents and executes 12 iterations, evaluating a total of 144 configurations.

Table 5.1 NinOA optimization results

Hypermeter	Bast Value
LSTM layers	3
Units per Layer	128
Dropout rate	0.5246
Optimizer	Adamw
Learning Rate	1.671×10^{-4}
Batch Size	32

5.3.5 Final Model Construction

Upon completion of optimisation, the best configuration is selected, and a final Long Short Term Memory model is constructed incorporating the optimal number of layers, units, dropout, and learning rate. This model is then trained extensively on the combined training and validation data, with performance evaluated on the held-out test set to assess generalisation.

After completion of optimisation, the best hypermeter configuration is selected and final Long Short Term Memory model is constructed using the optimised hypermeter configuration incorporating the optimal number of layers, units, dropout, and learning rate. This model is then trained on the combined training and validation data, and the performance evaluated on the test set (not used in any of the steps before, acting as unseen data) to assess the generalisation.

6. WORK DONE

6.1 Details of Implementation

The entire pipeline was implemented using Python programming language, leveraging TensorFlow and Keras for deep learning model construction and training.

Data preprocessing was performed using Pandas and NumPy libraries.

Scikit learn facilitated feature scaling and evaluation metrics computation.

The Ninja Optimisation Algorithm was custom implemented based on principles outlined in recent metaheuristic optimisation literature, with careful attention to handling both continuous and categorical hyperparameters.

All experiments were conducted on Google Colab with Graphics Processing Unit acceleration enabled to reduce training time.

Each hyperparameter configuration evaluation involved training a Long Short Term Memory model for up to 30 epochs, with early stopping to prevent unnecessary computation.

6.2 Results and Discussion

The Ninja Optimisation Algorithm successfully identified an optimal hyperparameter configuration that yielded superior performance compared to baseline models.

The optimised model achieved a validation Mean Absolute Error of 1.2128 and validation Root Mean Squared Error of 1.7302. On the independent test set, the model attained a Mean Absolute Error of 1.2237 and Root Mean Squared Error of 1.7357.

The training and validation loss curves for the Ninja Optimisation Algorithm optimised model exhibit steady convergence with minimal fluctuations, indicating stable learning dynamics and effective regularisation. The gap between training and validation loss is narrower compared to the baseline model, suggesting reduced overfitting and improved generalisation capability.

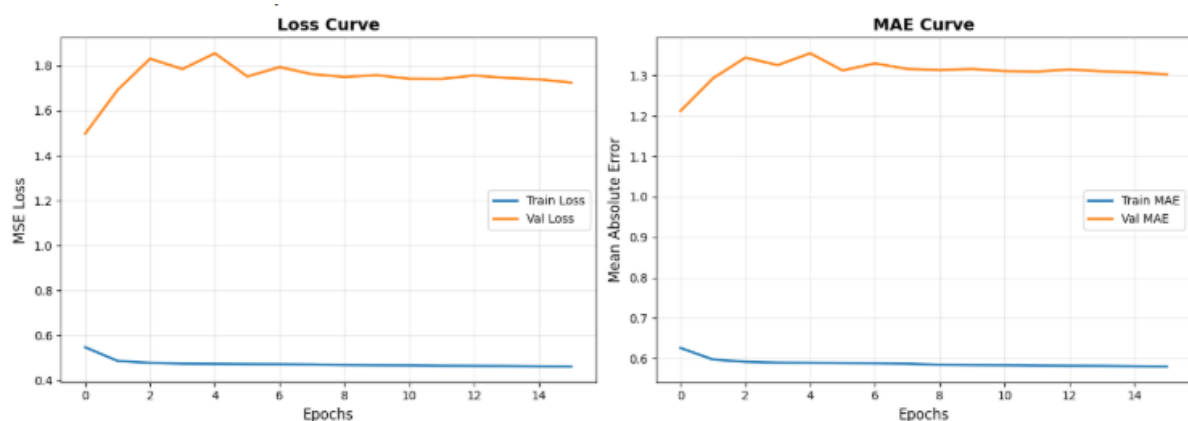


diagram 6.2.1 Loss curves

Convergence analysis of the Ninja Optimisation Algorithm reveals a monotonic decrease in fitness values over iterations, with the most significant improvements occurring in the initial

exploration phase. As the algorithm transitioned to exploitation, refinements led to incremental yet meaningful performance gains, ultimately converging to a near optimal solution.

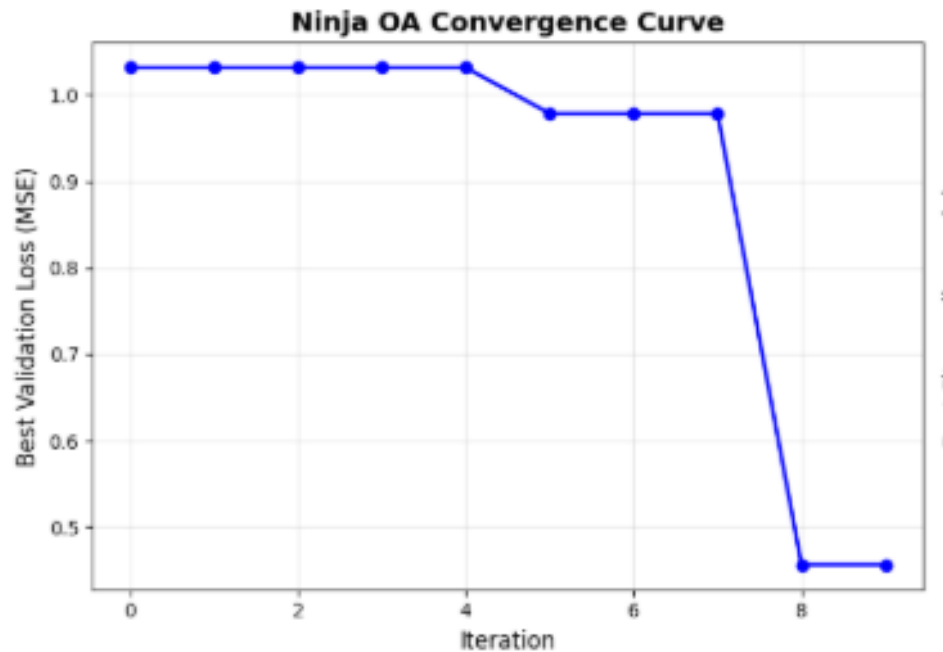


diagram 6.2.2 Ninja OA Convergence curve

```
lstm_layers: 3
units: 128
dropout: 0.5245520264652982
optimizer: adamw
learning_rate: 0.00016714717817375982
batch_size: 32
Model: "LSTM_NiOA"
```

diagram 6.2.3 NinOA Optimization results

Visual inspection confirms that the Ninja Optimisation Algorithm model maintains consistently lower validation errors throughout training epochs. Furthermore, the convergence curve of the Ninja Optimisation Algorithm itself demonstrates efficient search behaviour, validating the algorithm's adaptive phase selection mechanism.

6.3 Individual Contribution

This is an individual research project. All stages, from data acquisition, preprocessing, and feature engineering through model design, hyperparameter optimisation, training, evaluation, and documentation, were undertaken solely by the author of this report. The custom implementation of the Ninja Optimisation Algorithm and integration with the Long Short Term Memory model was conducted independently, drawing upon published literature and best practices in machine learning research.

7. CONCLUSION AND FUTURE PLAN

This project progress until now demonstrated the application of advanced deep learning techniques, specifically stacked and bidirectional Long Short Term Memory recurrent neural networks, for predicting carbon emissions in data centres.

By incorporating the Ninja Optimisation Algorithm for hyperparameter tuning, the project achieved measurable performance improvements over conventional optimisation approaches. The optimised model exhibits robust generalisation, low prediction errors, and stable training dynamics, making it a viable tool for real time energy monitoring and environmental impact assessment.

Future work will focus on several directions. Firstly, expanding the dataset to include additional data centres and longer observation periods will enhance model robustness and generalisability across diverse operational contexts.

Secondly, incorporating external variables such as weather conditions, electricity grid carbon intensity, and workload patterns may further improve prediction accuracy.

Thirdly, exploring ensemble methods that combine multiple optimised models could yield even more reliable forecasts.

Also, deploying the trained model as part of a real time monitoring system, integrated with data centre management software, will enable proactive energy management and carbon reduction strategies.

8. OUTCOMES

The project progress has resulted in a functional, well validated predictive model for carbon emission forecasting. Preliminary results have been documented, and a detailed technical report has been prepared for submission to academic professionals specialising in sustainable computing and machine learning applications. Efforts are underway to publish the findings in a peer reviewed journal, contributing to the growing body of knowledge on environmentally responsible computing infrastructure.

REFERENCES

- [1] Ben Ghorbal, A., Grine, A., Elbatal, I., Almetwally, E. M., Eid, M. M., & El-Kenawy, E. S. (2025). Predicting carbon dioxide emissions using deep learning and Ninja metaheuristic optimization algorithm. *Scientific Reports*, 15, Article 86251.
- [2] Yassen, M. A., El-Kenawy, E. M., Abdel-Fattah, M. G., Ismael, I., & Mostafa, H. E. S. (2025). Renewable energy forecasting using optimized quantum temporal model based on Ninja optimization algorithm. *Scientific Reports*, 15, Article 97109.
- [3] Ben Ghorbal, A., Grine, A., Eid, M. M., & El-Kenawy, E. M. (2025). Sustainable soil organic carbon prediction using machine learning and the ninja optimization algorithm. *Frontiers in Environmental Science*, 13, Article 1630762.
- [4] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.
- [5] Graves, A., & Schmidhuber, J. (2005). Framewise phoneme classification with bidirectional LSTM networks. *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing*, 5, V-877.